

多模态融合在小样本跨域场景下失效的原因与解决方案

VAL F1=1.0, TEST F1=0.18的根本原因是**"贪婪学习"**与跨设备域偏移的双重叠加。您的双分支融合网络正在经历两个致命问题：首先，多模态网络会**"贪婪地"**依赖单一模态而抑制另一模态；其次，255个小样本让模型轻易记住了设备特异性特征，这些特征在新变压器上完全失效。本报告提供理论分析和**可直接实施的PyTorch代码方案**。

融合失效的三重机制解析

多模态融合表现反而比单分支差，这一反直觉现象在学术界已有充分记录。ICML 2022年Wu等人的研究揭示了**"贪婪学习"**(Greedy Learning)**本质：多模态DNN会选择性地依赖降低训练损失最快的模态，而系统性地欠拟合**其他模态。** [NYU Scholars](#) [arXiv](#)

具体到您的CNN+MLP架构，假设CNN图像分支在早期训练中提供了更强的梯度信号，模型会逐渐将其作为**"主力"**，MLP分支的参数更新被边缘化。 [mlr](#) ICML 2025年Chaudhuri等人进一步发现了**"模态坍塌"**(Modality Collapse)****现象：融合层存在"秩瓶颈"**，当来自CNN的噪声特征与MLP的预测性特征在共享神经元中纠缠时，有用信息会被**"污染"**。 [openreview](#) [OpenReview](#)

小样本放大效应使问题雪上加霜。您的255个样本面对CNN特征空间(通常512-2048维)加MLP特征的高维假设空间，模型能够找到**数据集特异性捷径**——这些特征在训练集变压器上完美分离类别，但不具备物理意义，无法泛化。CREMA-D数据集上的实验显示，添加音频模态后准确率从59%**下降到**54%，正是因为多模态模型抓住了虚假相关性。 [Medium](#) [arXiv](#)

跨设备域偏移的非对称性是第三重打击。关键发现：不同模态经历的域偏移幅度不同——图像模态在跨域迁移后准确率下降**69.16%**，而音频模态仅下降42.96%。 [ResearchGate](#) 这意味着您的CNN分支和MLP分支在面对新变压器时**"漂移"**方向和幅度都不一致，导致融合层学到的权重组合失效。

模态Dropout：防止单模态主导的核心策略

模态Dropout是解决贪婪学习最直接有效的技术——训练时随机丢弃整个模态分支，强制模型学习每个模态的独立表示能力。 [Medium](#)

python

```

class ModalityDropout(nn.Module):
    """训练时随机丢弃整个模态，防止模态主导"""
    def __init__(self, p=0.3, use_learnable_tokens=True, feature_dims=(512, 128)):
        super().__init__()
        self.p = p
        self.use_learnable_tokens = use_learnable_tokens
        if use_learnable_tokens:
            # 可学习的"缺失模态"占位符，优于零向量
            self.cnn_missing = nn.Parameter(torch.randn(feature_dims[0]))
            self.mlp_missing = nn.Parameter(torch.randn(feature_dims[1]))

    def forward(self, cnn_feat, mlp_feat):
        if not self.training:
            return cnn_feat, mlp_feat

        batch_size = cnn_feat.size(0)
        drop_cnn = torch.rand(1).item() < self.p
        drop_mlp = torch.rand(1).item() < self.p

        # 确保至少保留一个模态
        if drop_cnn and drop_mlp:
            drop_cnn = torch.rand(1).item() > 0.5
            drop_mlp = not drop_cnn

        if drop_cnn:
            cnn_feat = self.cnn_missing.expand(batch_size, -1) if self.use_learnable_tokens \
                else torch.zeros_like(cnn_feat)
        if drop_mlp:
            mlp_feat = self.mlp_missing.expand(batch_size, -1) if self.use_learnable_tokens \
                else torch.zeros_like(mlp_feat)

```

```
return cnn_feat, mlp_feat
```

推荐参数：对于255样本的小数据集，设置 $p=0.3$ ，即30%概率丢弃某个模态。使用可学习token（`use_learnable_tokens=True`）效果优于简单置零，因为它提供了更有意义的"缺失信息"表示。

Emergent Mind

门控融合优于简单拼接

简单的特征拼接(concatenation)让模型自行决定如何组合两个模态，这在小样本下容易导致过拟合和模态失衡。*[arXiv](#)门控多模态单元(GMU)*显式学习模态权重，在小数据集上表现稳定。

openreview

python

```
class GatedMultimodalUnit(nn.Module):
    """GMU: 显式学习模态贡献权重"""
    def __init__(self, dim_cnn, dim_mlp, output_dim):
        super().__init__()
        self.cnn_transform = nn.Linear(dim_cnn, output_dim)
        self.mlp_transform = nn.Linear(dim_mlp, output_dim)
        self.gate = nn.Linear(dim_cnn + dim_mlp, output_dim)

    def forward(self, cnn_feat, mlp_feat):
        h_cnn = torch.tanh(self.cnn_transform(cnn_feat))
        h_mlp = torch.tanh(self.mlp_transform(mlp_feat))
        z = torch.sigmoid(self.gate(torch.cat([cnn_feat, mlp_feat], dim=-1)))
        # z接近1时偏向CNN，接近0时偏向MLP
        return z * h_cnn + (1 - z) * h_mlp
```

动态质量门控进一步增强鲁棒性：根据每个样本的输入质量动态调整模态权重。[ScienceDirect](#)这对于不同变压器数据质量可能不一致的场景尤为重要：

python

```
class DynamicQualityGating(nn.Module):
    """根据输入质量动态调整模态权重"""
    def __init__(self, dim_cnn, dim_mlp, hidden_dim=64):
        super().__init__()
        self.quality_cnn = nn.Sequential(
            nn.Linear(dim_cnn, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, 1), nn.Sigmoid()
        )
        self.quality_mlp = nn.Sequential(
            nn.Linear(dim_mlp, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, 1), nn.Sigmoid()
        )
        self.proj_cnn = nn.Linear(dim_cnn, hidden_dim)
        self.proj_mlp = nn.Linear(dim_mlp, hidden_dim)

    def forward(self, cnn_feat, mlp_feat):
        q_cnn = self.quality_cnn(cnn_feat)
        q_mlp = self.quality_mlp(mlp_feat)
        # 归一化门控权重
        total = q_cnn + q_mlp + 1e-8
        return (q_cnn/total) * self.proj_cnn(cnn_feat) + (q_mlp/total) * self.proj_mlp(mlp_feat)
```

跨设备域适应的三种实用方法

针对不同变压器之间的域偏移，推荐组合使用以下域适应损失：

方法一：MMD损失对齐特征分布

最大均值差异(MMD)通过最小化源域和目标域特征的分布差异来学习域不变特征：

python

```
def mmd_loss(source_features, target_features, kernel='rbf', sigma=1.0):
    """计算MMD损失，用于对齐不同设备的特征分布"""
    n_s, n_t = source_features.size(0), target_features.size(0)

    def rbf_kernel(x, y):
        dist = torch.cdist(x, y, p=2)
        return torch.exp(-dist**2 / (2 * sigma**2))

    K_ss = rbf_kernel(source_features, source_features)
    K_tt = rbf_kernel(target_features, target_features)
    K_st = rbf_kernel(source_features, target_features)

    mmd = K_ss.sum()/(n_s*n_s) + K_tt.sum()/(n_t*n_t) - 2*K_st.sum()/(n_s*n_t)
    return mmd
```

方法二：DANN对抗域判别器

域对抗神经网络通过**梯度反转层(GRL)**实现单阶段对抗训练： [IntechOpen](#)

python

```

class GradientReversalLayer(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, alpha):
        ctx.alpha = alpha
        return x.view_as(x)

    @staticmethod
    def backward(ctx, grad_output):
        return grad_output.neg() * ctx.alpha, None

class DomainDiscriminator(nn.Module):
    """域判别器：区分样本来自哪个设备"""
    def __init__(self, feature_dim, hidden_dim=128):
        super().__init__()
        self.discriminator = nn.Sequential(
            nn.Linear(feature_dim, hidden_dim),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(hidden_dim, 2) # 二分类：源域/目标域
        )

    def forward(self, features, alpha=1.0):
        reversed_features = GradientReversalLayer.apply(features, alpha)
        return self.discriminator(reversed_features)

```

方法三：CORAL协方差对齐

CORAL通过对齐二阶统计量（协方差矩阵）实现域适应，计算开销低：

```
python
```

```
def coral_loss(source_features, target_features):
    """CORAL: 对齐源域和目标域的协方差矩阵"""
    d = source_features.size(1)

    # 计算协方差矩阵
    source_centered = source_features - source_features.mean(dim=0)
    target_centered = target_features - target_features.mean(dim=0)

    source_cov = (source_centered.T @ source_centered) / (source_features.size(0) - 1)
    target_cov = (target_centered.T @ target_centered) / (target_features.size(0) - 1)

    return torch.sum((source_cov - target_cov)**2) / (4 * d * d)
```

组合损失函数的推荐配置：

```
python

# 总损失 = 分类损失 +  $\lambda_1$ *MMD +  $\lambda_2$ *CORAL +  $\lambda_3$ *域对抗损失
L_total = L_cls + 0.5 * L_mmd + 0.3 * L_coral + 1.0 * L_dann
```

编码器冻结与分阶段解冻策略

对于255样本的小数据集，**必须冻结预训练CNN编码器**的大部分层，否则会立即过拟合。

```
python
```



```

def setup_progressive_unfreezing(model, total_epochs):
    """分阶段解冻策略"""
    # 阶段1: 完全冻结CNN (前30%训练)
    for param in model.cnn_encoder.parameters():
        param.requires_grad = False

    # MLP分支可以训练
    for param in model.mlp_branch.parameters():
        param.requires_grad = True

    # 返回解冻调度
    return {
        int(total_epochs * 0.3): 'unfreeze_last_2_layers',
        int(total_epochs * 0.5): 'unfreeze_last_4_layers',
        int(total_epochs * 0.7): 'unfreeze_all'
    }

def apply_layerwise_lr(model, base_lr=1e-4):
    """层级学习率: 越深的层学习率越高"""
    params = [
        {'params': model.cnn_encoder.layer1.parameters(), 'lr': base_lr * 0.01},
        {'params': model.cnn_encoder.layer2.parameters(), 'lr': base_lr * 0.1},
        {'params': model.cnn_encoder.layer3.parameters(), 'lr': base_lr * 0.5},
        {'params': model.cnn_encoder.layer4.parameters(), 'lr': base_lr},
        {'params': model.mlp_branch.parameters(), 'lr': base_lr * 2},
        {'params': model.fusion.parameters(), 'lr': base_lr * 5},
        {'params': model.classifier.parameters(), 'lr': base_lr * 5},
    ]
    return torch.optim.AdamW(params, weight_decay=0.01)

```

抗过拟合的完整正则化配置

针对VAL F1=1.0的严重过拟合，推荐以下激进正则化组合：

技术	推荐值	说明
Dropout（隐藏层）	0.5-0.6	融合层使用更高值
模态Dropout	0.3	随机丢弃整个模态
Weight Decay	0.01-0.05	AdamW中设置
Label Smoothing	0.1-0.2	防止过度自信
Early Stopping	patience=15	监控验证集F1
梯度裁剪	max_norm=1.0	稳定训练

BatchNorm替换为LayerNorm： 对于小batch和跨域场景，BatchNorm的运行统计量不可靠，建议改用LayerNorm或GroupNorm：

```
python

# 替换前（不推荐）
self.bn = nn.BatchNorm1d(hidden_dim)

# 替换后（推荐）
self.ln = nn.LayerNorm(hidden_dim)
# 或对于CNN
self.gn = nn.GroupNorm(num_groups=8, num_channels=hidden_dim)
```

数据划分策略的关键错误纠正

您当前的VAL集可能存在**数据泄漏**：如果训练集和验证集包含来自同一设备的样本，模型会学到

设备特异性特征，在验证集上表现完美但无法泛化。

正确做法：Leave-One-Device-Out交叉验证

```
python

from sklearn.model_selection import LeaveOneGroupOut

def cross_device_evaluation(X, y, device_ids, model_fn):
    """留一设备交叉验证：获得真实的跨设备泛化性能"""
    logo = LeaveOneGroupOut()
    results = []

    for fold, (train_idx, test_idx) in enumerate(logo.split(X, y, groups=device_ids)):
        model = model_fn()

        # 训练在N-1个设备上
        train_loader = create_loader(X[train_idx], y[train_idx])
        # 测试在第N个设备上
        test_loader = create_loader(X[test_idx], y[test_idx])

        train_model(model, train_loader)
        score = evaluate(model, test_loader)

        results.append({
            'test_device': device_ids[test_idx[0]],
            'f1_score': score
        })

    return results
```

数据划分原则：

- 训练集：设备A、B、C的数据
- 验证集：设备D的数据（用于超参数调优和早停）
- 测试集：设备E的数据（最终评估，训练时完全不可见）

完整解决方案代码模板

```
python
```

```
class RobustMultimodalFaultDiagnosis(nn.Module):
    """整合所有改进的多模态故障诊断模型"""
    def __init__(self, num_classes, cnn_dim=512, mlp_input_dim=64,
                  mlp_hidden=128, fusion_dim=256):
        super().__init__()

        # 预训练CNN (初始冻结)
        self.cnn_encoder = models.resnet18(pretrained=True)
        self.cnn_encoder.fc = nn.Identity()

        # MLP分支 (可训练)
        self.mlp_branch = nn.Sequential(
            nn.Linear(mlp_input_dim, mlp_hidden),
            nn.LayerNorm(mlp_hidden), # 使用LayerNorm
            nn.ReLU(),
            nn.Dropout(0.4),
            nn.Linear(mlp_hidden, mlp_hidden),
            nn.LayerNorm(mlp_hidden),
            nn.ReLU(),
            nn.Dropout(0.4),
        )

        # 模态Dropout
        self.modality_dropout = ModalityDropout(p=0.3, feature_dims=(cnn_dim, mlp_hidden))

        # 门控融合
        self.fusion = GatedMultimodalUnit(cnn_dim, mlp_hidden, fusion_dim)

        # 域判别器 (用于DANN)
        self.domain_discriminator = DomainDiscriminator(fusion_dim)

        # 分类器
```

```

self.classifier = nn.Sequential(
    nn.Dropout(0.5),
    nn.Linear(fusion_dim, num_classes)
)

def forward(self, images, tabular, alpha=1.0, return_features=False):
    cnn_feat = self.cnn_encoder(images)
    mlp_feat = self.mlp_branch(tabular)

    cnn_feat, mlp_feat = self.modality_dropout(cnn_feat, mlp_feat)

    fused = self.fusion(cnn_feat, mlp_feat)
    class_output = self.classifier(fused)
    domain_output = self.domain_discriminator(fused, alpha)

    if return_features:
        return class_output, domain_output, fused
    return class_output, domain_output

def train_with_domain_adaptation(model, source_loader, target_loader, epochs=100):
    """带域适应的完整训练流程"""
    optimizer = apply_layerwise_lr(model, base_lr=1e-4)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingWarmRestarts(optimizer, T_0=20)
    criterion = nn.CrossEntropyLoss(label_smoothing=0.1)

    unfreeze_schedule = setup_progressive_unfreezing(model, epochs)
    early_stopper = EarlyStopping(patience=15, mode='max')

    for epoch in range(epochs):
        # 执行分阶段解冻
        if epoch in unfreeze_schedule:
            execute_unfreezing(model, unfreeze_schedule[epoch])

```

域适应系数递增

p = epoch / epochs

alpha = 2. / (1. + np.exp(-10 * p)) - 1

model.train()

for (src_img, src_tab, src_y), (tgt_img, tgt_tab, _) in zip(source_loader, target_loader):

源域前向

class_out, domain_out_s, feat_s = model(src_img, src_tab, alpha, return_features=True)

目标域前向

_, domain_out_t, feat_t = model(tgt_img, tgt_tab, alpha, return_features=True)

损失计算

loss_cls = criterion(class_out, src_y)

loss_domain = F.cross_entropy(domain_out_s, torch.zeros(src_img.size(0)).long()) + \
F.cross_entropy(domain_out_t, torch.ones(tgt_img.size(0)).long())

loss_mmd = mmd_loss(feat_s, feat_t)

loss_coral = coral_loss(feat_s, feat_t)

total_loss = loss_cls + loss_domain + 0.5*loss_mmd + 0.3*loss_coral

optimizer.zero_grad()

total_loss.backward()

torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

optimizer.step()

scheduler.step()

验证与早停

val_f1 = evaluate(model, val_loader)

if early_stopper(val_f1, model):

break

实施优先级与预期效果

按以下顺序逐步实施，每步验证效果后再进行下一步：

1. 立即实施（预期TEST F1提升至0.35-0.45）

- 添加模态Dropout ($p=0.3$)
- 替换BatchNorm为LayerNorm
- 将dropout从0.0/0.1提升到0.5

2. 短期改进（预期TEST F1提升至0.50-0.65）

- 将简单拼接替换为GMU门控融合
- 冻结CNN编码器前30%的训练轮次
- 添加Label Smoothing (0.1)

3. 域适应整合（预期TEST F1提升至0.65-0.80）

- 添加MMD损失对齐特征分布
- 实施DANN域对抗训练
- 使用Leave-One-Device-Out交叉验证

4. 高级优化（预期TEST F1提升至0.75-0.85+）

- 实施分阶段解冻策略
- 层级学习率配置
- 多模态Mixup数据增强 ([arXiv](#))

根据机械故障诊断领域的学术文献，结合MMD+CORAL+DANN的域适应方法在跨设备轴承故障诊断任务上能达到**95%+**的准确率。您的场景因样本更少且为变压器（特征可能更复杂）会略低，但上述方法组合应能显著改善当前0.18的TEST F1。

